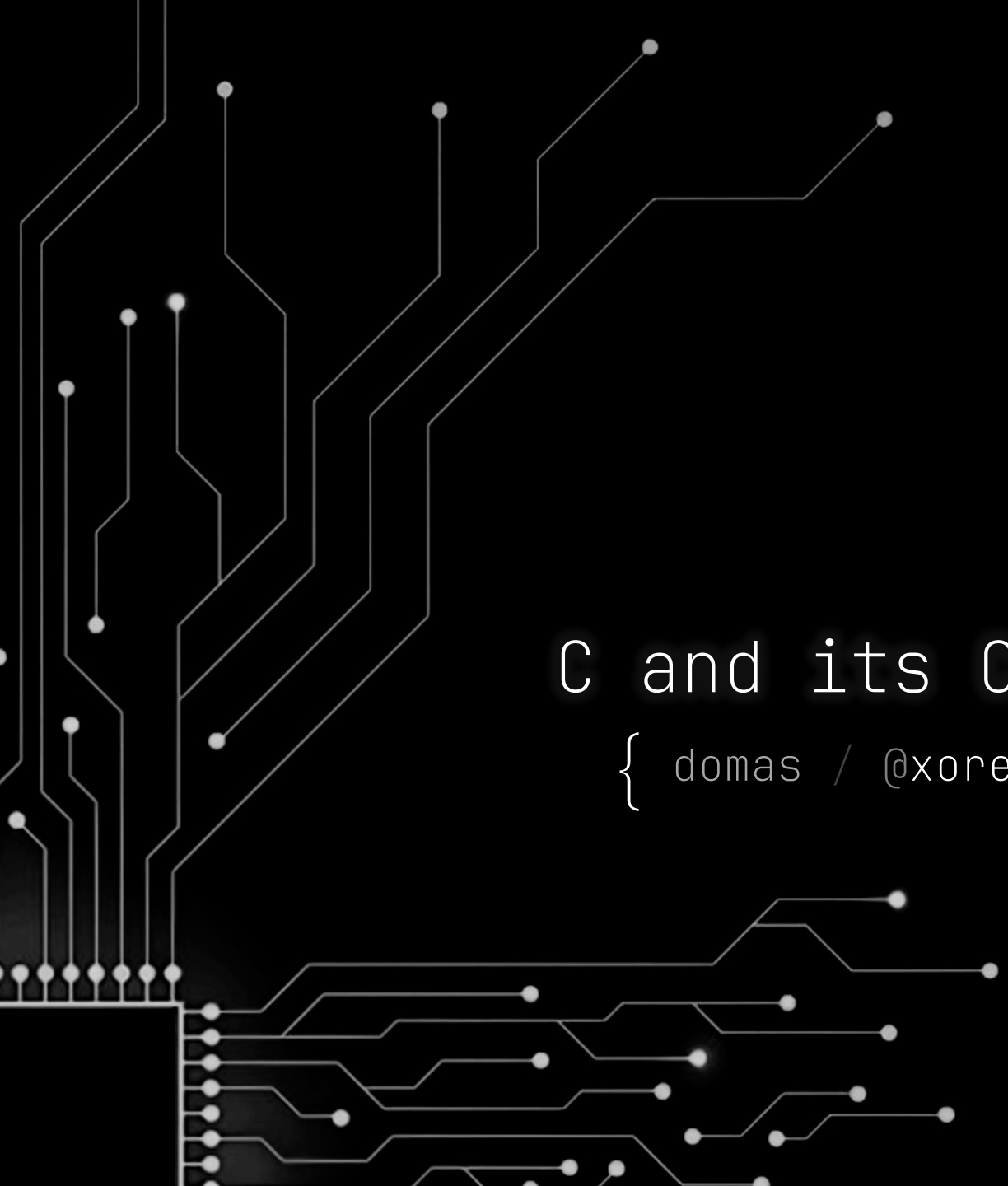


C and its Consequences

{ domas / @xoreaxeaxeax / Black Hat 2026

find the vulnerability.

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    if (pkt->length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, pkt->length);
    return 0;
}
```

TOCTOU

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    if (pkt->length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, pkt->length);
    return 0;
}
```

```
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    if (pkt->length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, pkt->length);
    return 0;
}
```

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    if (pkt->length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, pkt->length);
    return 0;
}
```

```
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    if (pkt->length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, pkt->length);
    return 0;
}
```

```
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    if (pkt->length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, pkt->length);
    return 0;
}
```


let’s fix it.

let’s fix it.

find the vulnerability.

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    /* copy size locally to prevent TOCTOU */
    unsigned int local_length = pkt->length;

    if (local_length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, local_length);
    return 0;
}
```

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    /* copy size locally to prevent TOCTOU */
    unsigned int local_length = pkt->length;

    if (local_length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, local_length);
    return 0;
}
```

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    /* copy size locally to prevent TOCTOU */
    unsigned int local_length = pkt->length;

    if (local_length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, local_length);
    return 0;
}
```

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    /* copy size locally to prevent TOCTOU */
    unsigned int local_length = pkt->length;

    if (local_length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, local_length);
    return 0;
}
```

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    /* copy size locally to prevent TOCTOU */
    unsigned int local_length = pkt->length;

    if (local_length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, local_length);
    return 0;
}
```

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    /* copy size locally to prevent TOCTOU */
    unsigned int local_length = pkt->length;

    if (local_length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, local_length);
    return 0;
}
```

no control over local = no attack window

~~TOCTOU~~

no control over local = no attack window

TOCTOU is impossible.

~~TOCTOU~~

“

... we’ve had compiler writers that say

‘if you read the specs, that’s ok’.

No, it’s not ok.

Because reality trumps any weasel-spec-reading.

”

— Linus Torvalds · lkml · 2019-08-16

C specifications

C specifications

- C ISO standard – ~700 pages of rules
- The *abstract machine*
  - A theoretical machine
  - What you're really programming in C
- Observable results
  - Output, file writes, external calls
  - Everything else is unwitnessed
- The *as-if* rule
  - Program on real machine must reproduce observable behavior
  - *as-if* it had run on the abstract machine
  - Nothing else has to match

- Compiler follows the _*as-if*_ rule
  - Owes only the visible _*result*_
  - The _*what*_ not the _*how*_
  - Doesn’t have to follow _*your*_ “how”
  - Unobservable can be reordered, merged, deleted

C specifications

- But what about our TOCTOU?
- We removed the dangerous reload
- Can the compiler undo this?
- Reintroduce the dangerous load?
- Check specification
- Spec says ... nothing.
- Loads aren't observable behavior
- Read once, twice, never — spec's indifferent
- If the spec is silent - compiler can do anything
- Only the final result is owed

C specifications

- _*invented load*_: extra reads, unrequested
- Nothing in C forbids them
- Could it _actually_ cause an "impossible" TOCTOU?

C specifications

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    /* copy size locally to prevent TOCTOU */
    unsigned int local_length = pkt->length;
    if (local_length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, local_length);
    return 0;
}
```

pkt->length

pkt->length

- Local copy would be in register.
- Replace two register reads with two memory reads.
- This would make code _slower_.
- An _optimizing_ compiler would never do this.

C specifications

“

...

if gcc did that, much of the kernel would go down in flames.

”

— Paul E. McKenney · lkml · 2009-04-16

proof-of-concept

- Idea seems nonsensical
- Source: read once, stash it in a register
- Re-reading memory is slower
- No reason to ever drop the local copy
- Compiler would never do this ...

proof-of-concept

- *...unless it runs out of registers*
- If every GPR is in use by compiler
- Compiler would need to _*spill*_ to the stack
- Spilling costs a store and a load
- Cheaper to just re-read the original
- The invented load is the optimization

proof-of-concept

- Can we make it happen on purpose?
- GCC 14.1
- Manufacture register pressure
- See if load reappears

proof-of-concept

```c
int x, y, z;

void f(void) {
    int t = x;
    y = t;
    z = t;
}
```

```asm
; gcc 14 -O0

mov     eax, DWORD PTR x[rip]
mov     DWORD PTR [rbp-4], eax

mov     eax, DWORD PTR [rbp-4]
mov     DWORD PTR y[rip], eax

mov     eax, DWORD PTR [rbp-4]
mov     DWORD PTR z[rip], eax
```

```c
int x, y, z;

void f(void) {
    int t = x;
    y = t;
    z = t;
}
```

```asm
; gcc 14 -O0

mov     eax, DWORD PTR x[rip]
mov     DWORD PTR [rbp-4], eax

mov     eax, DWORD PTR [rbp-4]
mov     DWORD PTR y[rip], eax

mov     eax, DWORD PTR [rbp-4]
mov     DWORD PTR z[rip], eax
```

```c
int x, y, z;

void f(void) {
    int t = x;
    y = t;
    z = t;
}
```

```asm
; gcc 14 -O0

mov     eax, DWORD PTR x[rip]
mov     DWORD PTR [rbp-4], eax

mov     eax, DWORD PTR [rbp-4]
mov     DWORD PTR y[rip], eax

mov     eax, DWORD PTR [rbp-4]
mov     DWORD PTR z[rip], eax
```

```c
int x, y, z;

void f(void) {
    int t = x;
    y = t;
    z = t;
}
```

```asm
; gcc 14 -O1

mov     eax, DWORD PTR x[rip]

mov     DWORD PTR y[rip], eax

mov     DWORD PTR z[rip], eax
```

```
int x, y, z;

void f(void) {
    int t = x;
    y = t;
    z = t;
}
```

```
; gcc 14 -O1

mov     eax, DWORD PTR x[rip]

mov     DWORD PTR y[rip], eax

mov     DWORD PTR z[rip], eax
```

```c
int x, y, z;

void f(void) {
    int t = x;
    y = t;
    z = t;
}
```

```asm
; gcc 14 -O1

mov     eax, DWORD PTR x[rip]

mov     DWORD PTR y[rip], eax

mov     DWORD PTR z[rip], eax
```

```c
#define CLOBBER \
    ""

int x, y, z;

void f(void) {
    int t = x;
    asm volatile("" : : : CLOBBER);
    y = t;
    asm volatile("" : : : CLOBBER);
    z = t;
}
```

```asm
; gcc 14 -O1

mov     eax, DWORD PTR x[rip]

mov     DWORD PTR y[rip], eax

mov     DWORD PTR z[rip], eax
```

```c
#define CLOBBER \
    ""

int x, y, z;

void f(void) {
    int t = x;
    asm volatile("" : : : CLOBBER);
    y = t;
    asm volatile("" : : : CLOBBER);
    z = t;
}
```

```asm
; gcc 14 -O1

mov     eax, DWORD PTR x[rip]

mov     DWORD PTR y[rip], eax

mov     DWORD PTR z[rip], eax
```

```c
#define CLOBBER \
    ""

int x, y, z;

void f(void) {
    int t = x;
    asm volatile("" : : : CLOBBER);
    y = t;
    asm volatile("" : : : CLOBBER);
    z = t;
}
```

```asm
; gcc 14 -O1

mov     eax, DWORD PTR x[rip]

mov     DWORD PTR y[rip], eax

mov     DWORD PTR z[rip], eax
```

- Doesn’t have to be inline asm
- Code, variables, calls, etc.
- Anything using registers will cause pressure

proof-of-concept

```c
#define CLOBBER \
    ""

int x, y, z;

void f(void) {
    int t = x;
    asm volatile("" : : : CLOBBER);
    y = t;
    asm volatile("" : : : CLOBBER);
    z = t;
}
```

```asm
; gcc 14 -O1

mov     eax, DWORD PTR x[rip]

mov     DWORD PTR y[rip], eax

mov     DWORD PTR z[rip], eax
```

```c
#define CLOBBER \
    "rax"

int x, y, z;

void f(void) {
    int t = x;
    asm volatile("" : : : CLOBBER);
    y = t;
    asm volatile("" : : : CLOBBER);
    z = t;
}
```

```asm
; gcc 14 -O1

mov     edx, DWORD PTR x[rip]

mov     DWORD PTR y[rip], edx

mov     DWORD PTR z[rip], edx
```

```c
#define CLOBBER \
    "rax","rdx"

int x, y, z;

void f(void) {
    int t = x;
    asm volatile("" : : : CLOBBER);
    y = t;
    asm volatile("" : : : CLOBBER);
    z = t;
}
```

```asm
; gcc 14 -O1

mov     ecx, DWORD PTR x[rip]

mov     DWORD PTR y[rip], ecx

mov     DWORD PTR z[rip], ecx
```

```c
#define CLOBBER \
    "rax","rdx","rcx"

int x, y, z;

void f(void) {
    int t = x;
    asm volatile("" : : : CLOBBER);
    y = t;
    asm volatile("" : : : CLOBBER);
    z = t;
}
```

```asm
; gcc 14 -O1

mov     esi, DWORD PTR x[rip]

mov     DWORD PTR y[rip], esi

mov     DWORD PTR z[rip], esi
```

```c
#define CLOBBER \
    "rax","rdx","rcx","rbx","rsi", \
    "rdi","rbp","r8","r9","r10",   \
    "r11","r12","r13","r14”

int x, y, z;

void f(void) {
    int t = x;
    asm volatile("" : : : CLOBBER);
    y = t;
    asm volatile("" : : : CLOBBER);
    z = t;
}
```

```asm
; gcc 14 -O1

mov     r15d, DWORD PTR x[rip]

mov     DWORD PTR y[rip], r15d

mov     DWORD PTR z[rip], r15d
```

```c
#define CLOBBER \
    "rax","rdx","rcx","rbx","rsi", \
    "rdi","rbp","r8","r9","r10",   \
    "r11","r12","r13","r14","r15"

int x, y, z;

void f(void) {
    int t = x;
    asm volatile("" : : : CLOBBER);
    y = t;
    asm volatile("" : : : CLOBBER);
    z = t;
}
```

```asm
; gcc 14 -O1

mov     eax, DWORD PTR x[rip]
mov     DWORD PTR y[rip], eax

mov     eax, DWORD PTR x[rip]
mov     DWORD PTR z[rip], eax
```

```c
#define CLOBBER \
    "rax","rdx","rcx","rbx","rsi", \
    "rdi","rbp","r8","r9","r10",   \
    "r11","r12","r13","r14","r15"

int x, y, z;

void f(void) {
    int t = x;
    asm volatile("" : : : CLOBBER);
    y = t;
    asm volatile("" : : : CLOBBER);
    z = t;
}
```

```asm
; gcc 14 -O1

mov     eax, DWORD PTR x[rip]
mov     DWORD PTR y[rip], eax

mov     eax, DWORD PTR x[rip]
mov     DWORD PTR z[rip], eax
```

- The as-if rule - only visible effects matter
- Compiler makes pragmatic choice
  - Eliminate local t, invent second load from x
  - 2 loads vs. 3 loads in -O0 code
- Confirmed feasible
  - Optimizing compiler
  - _can_ skip local value copy
  - Duplicate memory load _not in the source_
- Compiler invented loads
  - Not a vulnerability by itself
  - Almost always transparent
  - Unless...

proof-of-concept

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    /* copy size locally to prevent TOCTOU */
    unsigned int local_length = pkt->length;
    if (local_length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, local_length);
    return 0;
}
```

pkt->length

pkt->length

- ~~Impossible~~ TOCTOU

proof-of-concept

“

Insane? Probably so.
But there are compiler guys who swear by it.

”

— Paul E. McKenney · lkml · 2008-02-04

~~impossible~~ TOCTOU

- Mental model of what we’re _*supposed*_ to do
- Copy untrusted value to a local
- Validate the local — bounds check, etc.
- Use the local, assuming it can't change
- Compiler doesn’t care.

~~impossible~~ TOCTOU

- How do we get the compiler to listen?
- volatile constrains arbitrary reads/writes
- But unsettling issue...
- Nothing in the source _*asks*_ for it
- Widely omitted in the TOCTOU defense

~~impossible~~ TOCTOU

- Unexplored in security
- New TOCTOU class
- Negates standard TOCTOU-secure reasoning
- Vulnerability is not in the source
- Compiler decides, not the code
- Must compile to find out
- "Schrödinger's TOCTOU"

schrödinger's TOCTOU

“

Compilers that ‘optimize’ things
to touch fields that aren’t touched by the source code
are simply inherently buggy shit.

”

— Linus Torvalds · lkml · 2014-12-04

semantic drift

- PoC: compiler _*can*_ emit "impossible" TOCTOU
- But how often, in real code?
- When do invented loads _*really*_ happen?

semantic drift

- Source says: copy to a local
- Transform is legal - copy disappears
- Binary re-reads untrusted memory
- Intent ≠ behavior → *semantic drift*

semantic drift

- Review compiler source to find culprit?
- Frontend → IR → regalloc → codegen
- No clear single place responsible
- Readable source ≠ explainable output
- Treat compiler as black-box
- RE compilation pipeline's _*emergent behavior*_
- Characterize when Schrödinger TOCTOU occurs

semantic drift

static RE

the cat-states

- Manual trial and error
- Find simple code patterns
  that emit an invented load
- A “cat-state”: minimal example of some
  mechanism triggering invented load

```c
extern int opaque(int a);
extern const int x;

int y, z, w;

int g(int a) {
    int t = x; // one read of x
    w = t;
    /* a dozen opaque() calls */
    y = t;
    z = t;
    return 0;
}
```

```asm
; x86-64 gcc -O2
;   (also: icx, icc, clang, msvc)

g:
    mov     edx, DWORD PTR x[rip]
    mov     DWORD PTR w[rip], edx
    …
    mov     eax, DWORD PTR x[rip]
    mov     DWORD PTR y[rip], eax
    mov     DWORD PTR z[rip], eax
    …
```

cat-state: rematerialization

```c
unsigned int g(unsigned short *p)
{
    short t = *p; // one read of *p
    return t < 0 ? 0u : (unsigned short)t;
}
```

```asm
; ARM gcc -O2
;   (also: MIPS, -Og, -O1, -O2, -O3)

f:
    …
    ldrsh   r2, [r3]
    ldrh    r0, [r3]
    cmp     r2, #0
    it      lt
    movlt   r0, #0
    bx      lr
```

cat-state: width-mismatch reload

```c
typedef struct {
    int  len;
    char data[12];
} S;

S dst;

int g(S *p) {
    S u = *p; // one read of *p
    if (u.len > 0) {
        dst = u;
        return 1;
    }
    return 0;
}
```

```asm
; x86-64 gcc -O2
;   (also aarch64, ppc64, mips64,
;    s390x, -O1/-O2/-O3/-Os)

g:
    mov     edx, DWORD PTR [rdi]
    movdqu  xmm0, XMMWORD PTR [rdi]
    test    edx, edx
    jg      .L7
    ret
.L7:
    movaps  XMMWORD PTR dst[rip], xmm0
    mov     eax, 1
    ret
```

cat-state: bulk-vs-scalar overlap

```c
typedef union {
    float as_float;
    long long as_int;
} v_t;

v_t x;

long long g(v_t *p) {
    v_t u = *p; // one read of *p
    if (u.as_float + 1.0f > 0.0f)
        return u.as_int;
    return 0;
}
```

```asm
; x86-64 gcc -O2
;   (also: s390x)

g:
    …
    addss   xmm0, DWORD PTR [rdi]
    mov     rax, QWORD PTR [rdi]
    comiss  xmm0, xmm1
    cmovbe  rax, rdx
    ret
```

cat-state: cross-class reload

variant analysis

```c
typedef struct {
    char target;
    char rest[16];
} S;

S dst;
int sink;

void g(S *p) {
    S u;
    memcpy(&u, p, sizeof(u)); // one read of *p
    sink = u.target * 3;
    dst  = u;
}
```

```asm
; x86-64 gcc -O2
;   (also: aarch64, -O1/-O2/-O3/-Os)

g:
    movsx   eax, BYTE PTR [rdi]
    movdqu  xmm0, XMMWORD PTR [rdi]
    mov     rdx, QWORD PTR [rdi+16]
    lea     eax, [rax+rax*2]
    movaps  XMMWORD PTR dst[rip], xmm0
    mov     DWORD PTR sink[rip], eax
    mov     QWORD PTR dst[rip+16], rdx
    ret
```

memcpy

```c
typedef struct {
    char target;
    char rest[16];
} S;

S dst;
int sink;

void g(volatile S *p) {
    S u;
    memcpy(&u, p, sizeof(u)); // one read of *p
    sink = u.target * 3;
    dst  = u;
}
```

```asm
; x86-64 gcc -O2
;   (also: aarch64, -O1/-O2/-O3/-Os)

g:
    movsx   eax, BYTE PTR [rdi]
    movdqu  xmm0, XMMWORD PTR [rdi]
    mov     rdx, QWORD PTR [rdi+16]
    lea     eax, [rax+rax*2]
    movaps  XMMWORD PTR dst[rip], xmm0
    mov     DWORD PTR sink[rip], eax
    mov     QWORD PTR dst[rip+16], rdx
    ret
```

volatile

```c
// module_1.c
typedef struct { int  len; char data[12]; } S;
S dst;

int  ok (const S *q);
void use(const S *q);

int g(S *p) {
    S u = *p;       // one read of *p
    if (ok(&u))     // other TU - opaque
        use(&u);    // other TU - opaque
    return 0;
}
```

```c
// module_2.c
int  ok (const S *q) { return q->len > 0; }
void use(const S *q) { dst = *q; }
```

```asm
; x86-64 gcc -O2 -flto
;   (also: aarch64, ppc64, mips64, s390x)

g:
        mov     edx, DWORD PTR [rdi]
        movdqu  xmm0, XMMWORD PTR [rdi]
        test    edx, edx
        jg      .L7
        ret
.L7:
        movaps  XMMWORD PTR dst[rip], xmm0
        mov     eax, 1
        ret
```

translation units

- ~50 distinct cat-states
- Manufactured across
  at least six independent compiler subsystems
- Vulnerable vs. not-vulnerable depends on:
  - compiler×version×architecture×flags
  - register pressure
  - structure layout
  - type width & signedness
  - float⇄int unions
  - byte-order conversions
  - auto-vectorization
  - sub-word atomics
  - CISC memory-operand folds

manual RE results

- Static RE is insufficient

problem

dynamic RE

- Have initial datapoints, show _*can*_ occur
- Want to know _*everywhere*_
- Goal: concretely resolve the boundaries of semantic drift
- *Exact* how/when/why a given code shape emits a TOCTOU
  - compilers? code patterns? flags?
- Then defeating should be feasible

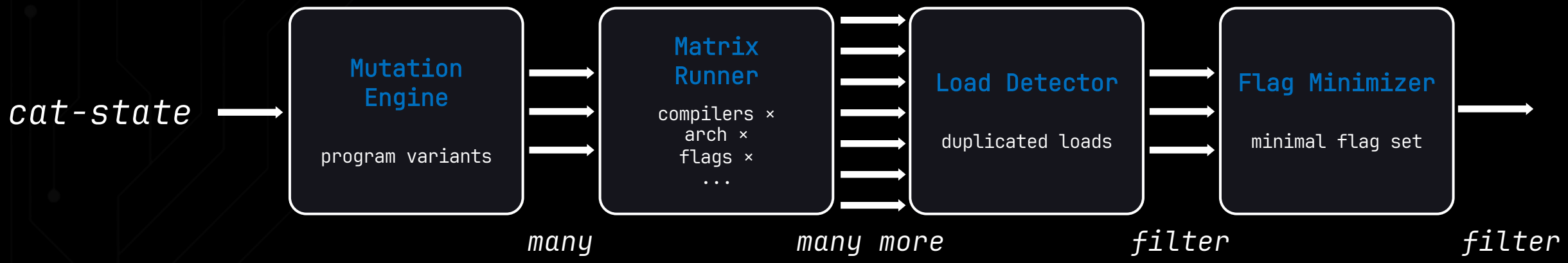
a semantic *fuzzer*

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    /* copy size locally to prevent TOCTOU */
    unsigned int local_length = pkt->length;

    if (local_length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, local_length);
    return 0;
}
```


cat-state
Mutation Engine
program variants
Matrix Runner
compilers ×
arch ×
flags ×
...
Load Detector
duplicated loads
Flag Minimizer
minimal flag set
many
many more
filter
filter

alpha-lab

- systemic mutations of cat-states
- struct fields = explore offset dependencies
- array size = block copy optimizations
- local variables = impact register pressure
- depth of function calls = function inlining
- generate hundreds of mutations

mutation engine

- Local CE instance / 200 GB of compilers
- Intake code from mutation engine
- Sweep compiler×version×architecture×flags
  - Compiler: 5 toolchains: GCC, Clang, ICC, ICX, MSVC
  - Version: 200+ builds, ~20 years of releases
  - Arch: x86-64, arm64, arm, RISC-V 32/64, m68k, MSP430
  - Flags: opt × lto × fp × pic × pie × stack
- 10,000+ compilations for each input

matrix runner

- 1M+ outputs from matrix runner
- Identify whether invented load exists
- Obvious
  - [r0] followed by [r0]
- Non-obvious
  - [edx] vs. [esi]
  - Loops
  - Conditionals
  - Architectures

load detector

load detector

- Unicorn emulator
- cat-state specifies C variable “x”
- Unicorn traces memory accesses
- Detect multiple reads from “x”
- Auto-detect invented loads from matrix runner
- Down-select to only invented load inputs

- Thousands of outputs from load detector
- Mostly duplicates
- Which caused the double-load?
- What *combinations?*
- ~250 optimizer flags, ~10^65 configurations
- Delta debugging minimization
- Additive sweep (-O0, -f…)
- Subtractive sweep (-O3, -fno…)

flag minimizer

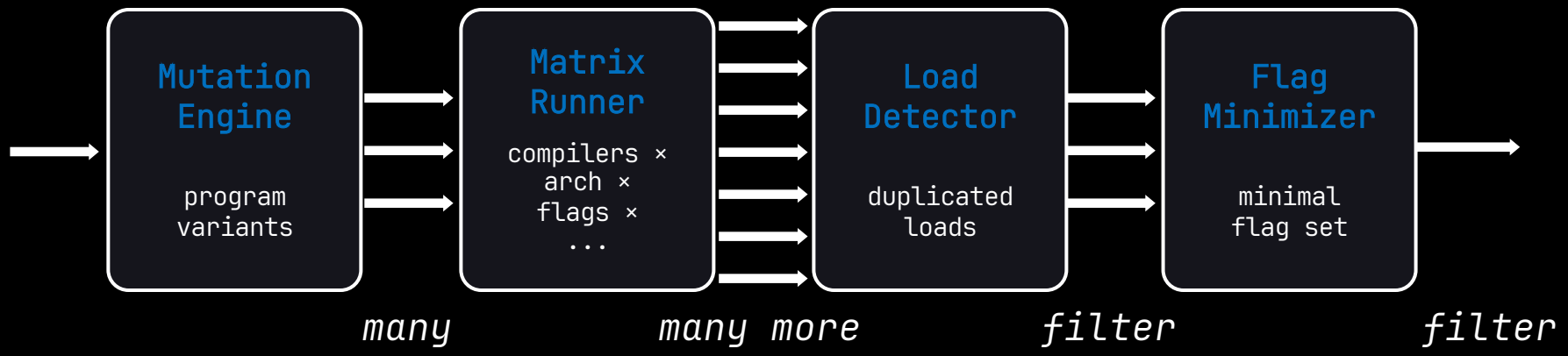
dynamic RE

- Example:
- Start with one cat-state
- Feed to the alpha-lab pipeline
- Characterize the _*boundary conditions*_
- Defeat Schrödinger's TOCTOU?

```c
typedef struct {
    int  len;
    char data[12];
} S;

S dst;

void g(S *p) {
    S u = *p;
    if (u.len > 0)
        dst = u;
}
```



/* safe/vuln changes at size threshold */

```c
typedef struct {
    int target;
    int rest[33];
} S;

S dst;

void g(S *p) {
    S u = *p;
    if (u.target > 0) dst = u;
}
```

```c
typedef struct {
    int target;
    int rest[34];
} S;

S dst;

void g(S *p) {
    S u = *p;
    if (u.target > 0) dst = u;
}
```

```asm
; SAFE — sizeof(S) == 136 (target read once)

g:
    mov     edx, DWORD PTR [rdi]
    test    edx, edx
    jle     .L1
    movd    xmm4, DWORD PTR [rdi+28]
    ...
.L1:
    mov     eax, ecx
    ret
```

```asm
; VULN — sizeof(S) == 140 (target read twice)

g:
    mov     eax, DWORD PTR [rdi]
    movdqu  xmm7, XMMWORD PTR [rdi]
    ...
    test    eax, eax
    jg      .L7
    ...
.L7:
    movaps  XMMWORD PTR dst[rip], xmm7
    ...
    ret
```

/* safe/vuln changes at field order */

```c
typedef struct {
    int target;
    int rest[5];
} S;

S dst;

void g(S *p) {
    S u = *p;
    if (u.target > 0) dst = u;
}
```

```c
typedef struct {
    int head[5];
    int target;
} S;

S dst;

void g(S *p) {
    S u = *p;
    if (u.target > 0) dst = u;
}
```

```asm
; SAFE — accessed field is first

g:
    mov     eax, DWORD PTR [rdi]
    test    eax, eax
    jle     .L1
    ...
    movd    xmm0, eax
    mov     rdx, QWORD PTR [rdi+16]
    ...
.L1:
    ret
```

```asm
; VULN — accessed field is second

g:
    mov     eax, DWORD PTR [rdi+20]
    test    eax, eax
    jle     .L1
    mov     rax, QWORD PTR [rdi+16]
    movdqu  xmm0, XMMWORD PTR [rdi]
    ...
.L1:
    ret
```

/* safe/vuln changes on compiler version */

```c
struct S {
    unsigned a : 3;
    unsigned b : 29;
};

struct S x;

int f(void) {
    struct S u = x;
    return u.a ? u.b : 0;
}
```

```asm
; RISC-V64, gcc 14.3+ — SAFE (x read once)

f:
    lui     a5, %hi(x)
    lw      a5, %lo(x)(a5)
    li      a0, 0
    andi    a4, a5, 7
    beq     a4, zero, .L2
    srliw   a0, a5, 3
.L2:
    ret
```

```asm
; RISC-V64, gcc 14.1 / 14.2 — VULN (x read twice)

f:
    lui     a4, %hi(x)
    lw      a5, %lo(x)(a4)
    li      a0, 0
    andi    a5, a5, 7
    beq     a5, zero, .L2
    ld      a0, %lo(x)(a4)
    srliw   a0, a0, 3
.L2:
    ret
```

/* safe/vuln changes on flags */

```c
typedef struct {
    int len;
    char data[12];
} S;   /* 16 B */

S dst;

int g(S *p) {
    S u = *p;
    dst = u;
    return u.len;
}
```

```asm
; gcc -O2 -fno-tree-sra (SRA off)
;   SAFE (len read once)
g:
    movdqu  xmm0, XMMWORD PTR [rdi]
    movd    eax, xmm0
    movaps  XMMWORD PTR dst[rip], xmm0
    ret
```

```asm
; gcc -O2 (SRA on)
;   VULN (len read twice)
g:
    movdqu  xmm0, XMMWORD PTR [rdi]
    mov     eax, DWORD PTR [rdi]
    movaps  XMMWORD PTR dst[rip], xmm0
    ret
```

/* safe/vuln changes on architecture tune */

```c
typedef struct {
    int target;
    int rest[19];
} S;

S dst;

int g(S *p) {
    S u = *p;
    dst = u;
    return u.target;
}
```

```asm
; gcc -O2 -mtune=generic
; → SAFE (target read once)
g:
    movdqu  xmm0, XMMWORD PTR [rdi]
    ...
    movd    eax, xmm0
    ret
```

```asm
; gcc -O2 -mtune=znver4
; → VULN (target read twice)
g:
    movdqu  xmm4, XMMWORD PTR [rdi]
    mov     eax, DWORD PTR [rdi]
    ...
    ret
```

/* safe/vuln changes on sizeof ≡ 1 (mod 16) */

```c
typedef struct {
    char head[16];
    char t;
} S;

S dst;

char g(S *p) {
    S u = *p;
    dst = u;
    return u.t;
}
```

```asm
; sizeof = 17
; → SAFE (t read once)
g:
    ; bytes 0..15  (not t)
    movdqu  xmm0, XMMWORD PTR [rdi]
    movzx   eax, BYTE PTR [rdi+16]
    movaps  XMMWORD PTR dst[rip], xmm0
    mov     BYTE PTR dst[rip+16], al
    ret
```

```c
typedef struct {
    char head[17];
    char t;
} S;

S dst;

char g(S *p) {
    S u = *p;
    dst = u;
    return u.t;
}
```

```asm
; sizeof = 18
;  -> VULN (t read twice)
g:
    movdqu  xmm0, XMMWORD PTR [rdi]
    movzx   edx, WORD PTR [rdi+16]
    movzx   eax, BYTE PTR [rdi+17]
    mov     WORD PTR dst[rip+16], dx
    ret
```

Compiler	cat-states	Architectures
GCC	• Rematerialization • Width-mismatch reload • Bulk-vs-scalar overlap • Cross-class reload • CISC mem-op fold • Byte-order reload	x86-64 · i386 · ARM · AArch64 · MIPS · MIPS64 · RV32 · RV64 · LoongArch64 · PPC64, SPARC · s390x · m68k · VAX · MSP430 · AVR · HPPA · Xtensa
Clang	• Rematerialization • Bulk-vs-scalar • CISC mem-op fold	x86-64 · AArch64 · PPC64 · MIPS64 · RV64 · RV32 · 6502
ICX	• Rematerialization • Bulk-vs-scalar	x86-64
ICC	• Rematerialization	x86-64
MSVC	• Rematerialization • Bulk-vs-scalar	x86-64

TOCTOU by compiler

Architecture	ISA property that invites them
x86-64	register-rich, cheap RIP-relative global reload, split FP/GPR files
i386	single-instruction absolute global reload + register-poor 8-GPR file; bulk copy overlaps a scalar field load
s390x	signed+unsigned 32→64 widening loads lgf/algf; an FP/GPR split, memory-operand ALU, and a byte-reversed load
ARM (32-bit family)	rich narrow-load variants + pipelined loads; ldm bulk copy overlaps a scalar ldr
AArch64	wide ldp/ldr q bulk copy and NEON ld2 de-interleave overlap a scalar field load (3); adrp+ldr addressing and free extend operand-modifiers suppress 1 and 2
MIPS	rich narrow loads, like ARM; word-granular atomics word-load a neighbor on MIPS64
RISC-V (RV64)	lw+ld bitfield reload; wide ld bulk copy vs scalar lw
LoongArch64	wide ldptr.d bulk copy vs scalar ldptr.w field load
PPC64	no unaligned vector load — a realigned wide load reloads each straddling aligned block
SPARC	word-granular atomics only — a sub-word _Atomic RMW word-loads a neighbor via an ld+cas loop
m68k / MSP430 / VAX	single-instruction global addressing; CISC memory-operand ALU add.l x,%d0
6502	memory-operand ALU; the one non-GCC instance

TOCTOU by architecture

- struct size/shape (17B/byte, 140B/dword)
- struct tail, field position (first vs. last)
- sizeof ≡ 1 (mod 16)
- Direction (4 → 8B = 2 reads, 8 → 4B = 1 read)
- Optimization level (-O2 vs. -Os vs. -Og)
- Flags (-O2 vs. -ftree-sra vs. "-fcode-hoisting + -ftree-ccp + -ftree-forwprop + -ftree-fre + -ftree-pre + -ftree-vrp")
- Compiler version (gcc 14.3 vs 16.1)
- ISA extensions (512-bit zmm or RISC-V +zicond)
- Host CPU (-mtune)

- vulnerable or safe can depend on
  which machine compiled the code
  - e.g. single read on Intel, double on AMD
- compiler bump can turn safe into vulnerable
  - e.g. MSVC 19.51, Clang 15; GCC 14.3 / 16.1 the other way
- safety hangs on details that mean nothing
  - e.g. one byte of struct size, sizeof mod 16,
    a trailing char[], field order,
    even which direction a union widens
  - each silently flips safe to vulnerable
- there is no single switch to turn it off
- spans the whole ecosystem
- search space is effectively unbounded

alpha-lab conclusions

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    /* copy size locally to prevent TOCTOU */
    unsigned int local_length = pkt->length;

    if (local_length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, local_length);
    return 0;
}
```

alpha-lab conclusions

- alpha-lab conclusion: *cannot* predict outcome
- No longer: _*does*_ a compiler do this
- Now: _*can*_ a compiler do this
- Schrödinger pattern = _*de facto vulnerable*_

“

When you have to go read the compiler sources
to figure things like this out,
you know you are too deep.

”

— Linus Torvalds · lkml · 2021-09-13

attack surface

- Where should we search?
- Anywhere data crosses a trust boundary
  and stays writable by the untrusted side

attack surface

- User → kernel
  - syscall args, copy_from_user snapshots, io_uring shared rings
- Guest → host VMM
  - virtio rings, device emulation (AHCI/NVMe descriptors)
- Malicious host → confidential guest
  - SEV-SNP, TDX, Arm CCA; shared/bounce buffers
- Secure world & enclaves
  - SMM/SMI, TrustZone, SGX/TEE, EL3 monitor
- Devices & DMA
  - descriptor rings, MMIO, peripheral-writable buffers
- Coprocessors over shared DRAM
  - rpmsg/remoteproc, SCP/PSP, mailboxes, cross-VM shmem
- Untrusted-format parsers
  - mmap'd files, fonts/images/archives, on-disk DBs, IPC
- Network / wire protocols
  - RDMA, NVMe-oF, MCTP/PLDM/SPDM, NTP

- Search to find Schrödinger pattern
  (snapshot → validate → use)
- ... where the C-specification _*allows*_ a TOCTOU
- Not every case is susceptible to invented-TOCTOU
- Certain barriers prevent emitting the second load

attack surface

- Example: volatile

barriers

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    /* copy size locally to prevent TOCTOU */
    unsigned int local_length = pkt->length;

    if (local_length > MAX_SIZE) {
        return -1;
    }

    memcpy(buffer, pkt->data, local_length);
    return 0;
}
```

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    /* copy size locally to prevent TOCTOU */
    unsigned int local_length = pkt->length;

    if (local_length > MAX_SIZE) {
        return -1;
    }

    memcpy(buffer, pkt->data, local_length);
    return 0;
}
```

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    /* copy size locally to prevent TOCTOU */
    unsigned int local_length =
        *(volatile unsigned int *)&pkt->length;

    if (local_length > MAX_SIZE) {
        return -1;
    }

    memcpy(buffer, pkt->data, local_length);
    return 0;
}
```

- volatile — at the exact access site
- atomic — pins + orders
- "memory"-clobber barrier
- opaque copy — asm/.S primitive
- copy-then-unmap / physical-address-handle
- read-only mapping
- per-tenant encryption (TDX/SEV/CCA)

barriers

- This doesn’t scale
- Even for _*small*_ snippets of code
- Outsourced to LLM-driven audit harness
- “observer-effect”

observer-effect

- Agent picks security-relevant open-source project
- Syncs state and goals with other agents
- Pulls source
- Searches attacker-writable trust boundaries
- Finds Schrödinger pattern: snapshot → validate → use
- Evaluates: does C spec *_permit_* the invented load?
- Traces data flow to leaf for barriers
- Tracks memories, mistakes, progress
- Ranks impact, adversarial re-review, emit report

observer-effect

- Point at range of security-critical open-source
- Run for ~100 hours

observer-effect

“

People love to talk about ‘safe C’,
but compiler people have
actively tried to make C unsafer for decades.
The C standards committee has been complicit.

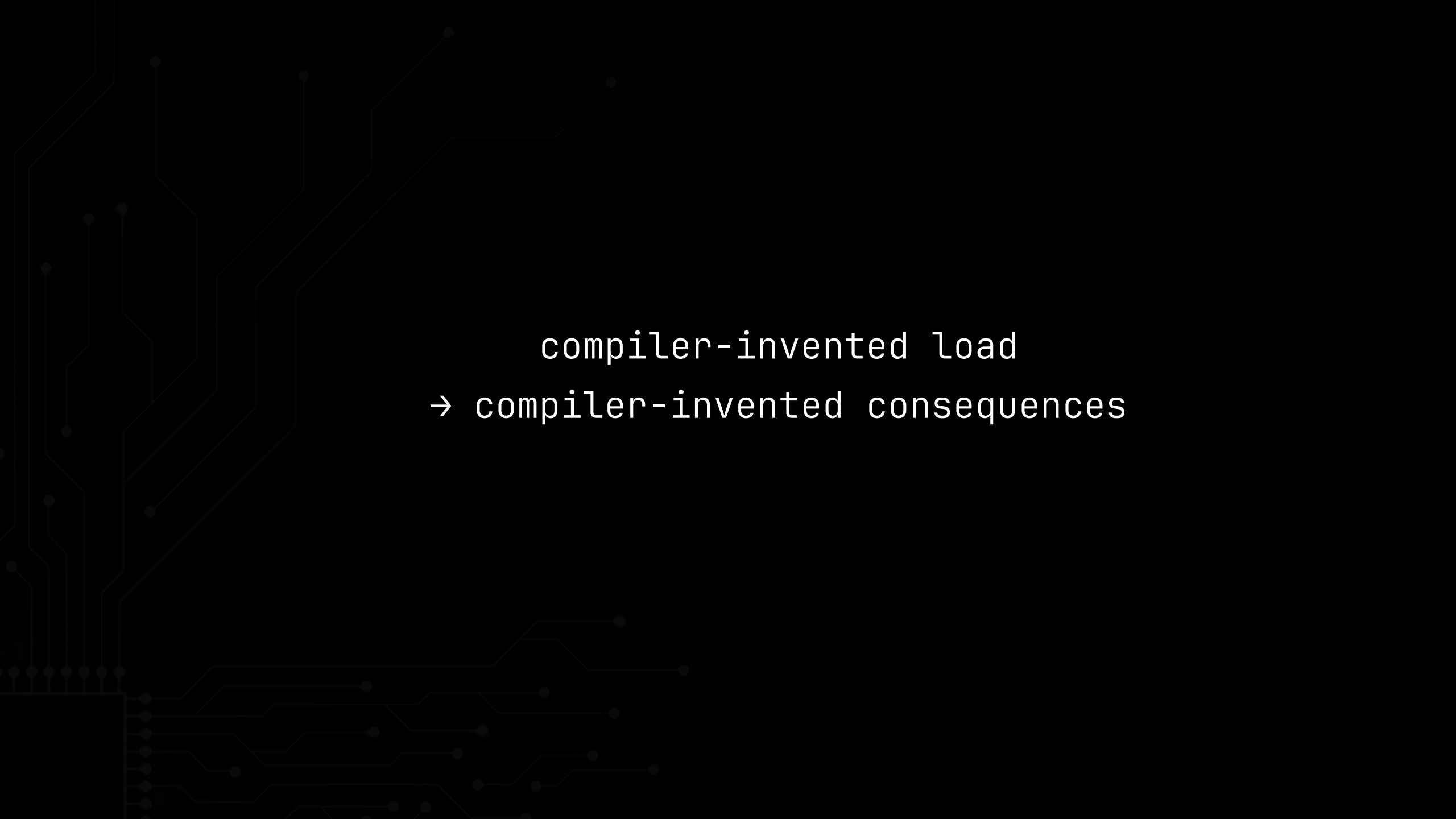
”

— Linus Torvalds · lkml · 2025-02-21

impact

100+ security-critical projects

300+ Schrödinger TOCTOUs


compiler-invented load
→ compiler-invented consequences


compiler-invented VM escapes

// qemu

```
uint16_t prdtl = le16_to_cpu(cmd->prdtl);          // 907  [SNAPSHOT] one read of guest header
dma_addr_t prdt_len = (prdtl * sizeof(AHCI_SG));   // 910  use #1: size the PRDT mapping
dma_addr_t real_prdt_len = prdt_len;               // 911
/* ... */
if (!(prdt = dma_memory_map(ad->hba->as, prdt_addr, &prdt_len,   // 929  map prdtl×16 bytes
                            DMA_DIRECTION_TO_DEVICE, MEMTXATTRS_UNSPECIFIED))){ /*...*/ }
if (prdt_len < real_prdt_len) {                    // 936  [CHECK] confirm full region mapped
    /* ... */ goto out;                            //        (bound tied to prdtl-at-910)
}
if (prdtl > 0) {
    AHCI_SG *tbl = (AHCI_SG *)prdt;
    for (i = 0; i < prdtl; i++) {                  // 948  [USE] walk bound over the mapping
        tbl_entry_size = prdt_tbl_entry_size(&tbl[i]);     //     reads tbl[i] (mapped guest mem)
        /* ... */
    }
    qemu_sglist_init(sglist, qbus->parent, (prdtl - off_idx), ad->hba->as);  // 964 [USE] alloc hint
    for (i = off_idx + 1; i < prdtl && sglist->size < limit; i++) {          // 970 [USE] walk bound
        qemu_sglist_add(sglist, le64_to_cpu(tbl[i].addr), /*...*/);          // 971 OOB entry → DMA
    }
}
```


compiler-invented root

// linux/rdma

```
 rqe = &qp->recvq[qp->rq_get % qp->attrs.rq_size];  // 349  rqe -> userspace-shared
}                                                   //      vmalloc_user recvq slot
if (likely(rqe->flags == SIW_WQE_VALID)) {          // 351
    int num_sge = rqe->num_sge;                     // 352  [SNAPSHOT] one read

    if (likely(num_sge <= SIW_MAX_SGE)) {           // 354  [CHECK] num_sge <= 6
        int i = 0;

        wqe = rx_wqe(&qp->rx_untagged);             // 357  kernel-private siw_wqe
        rx_type(wqe) = SIW_OP_RECEIVE;
        wqe->wr_status = SIW_WR_INPROGRESS;
        wqe->bytes = 0;
        wqe->processed = 0;

        wqe->rqe.id = rqe->id;
        wqe->rqe.num_sge = num_sge;                 // 364
        while (i < num_sge) {                       // 366  [USE] bound -> fixed sge[6]/mem[6]
            wqe->rqe.sge[i].laddr = rqe->sge[i].laddr; // 367  compiler may re-derive
            wqe->rqe.sge[i].lkey = rqe->sge[i].lkey;   //      `num_sge` from live rqe
            wqe->rqe.sge[i].length = rqe->sge[i].length;
```

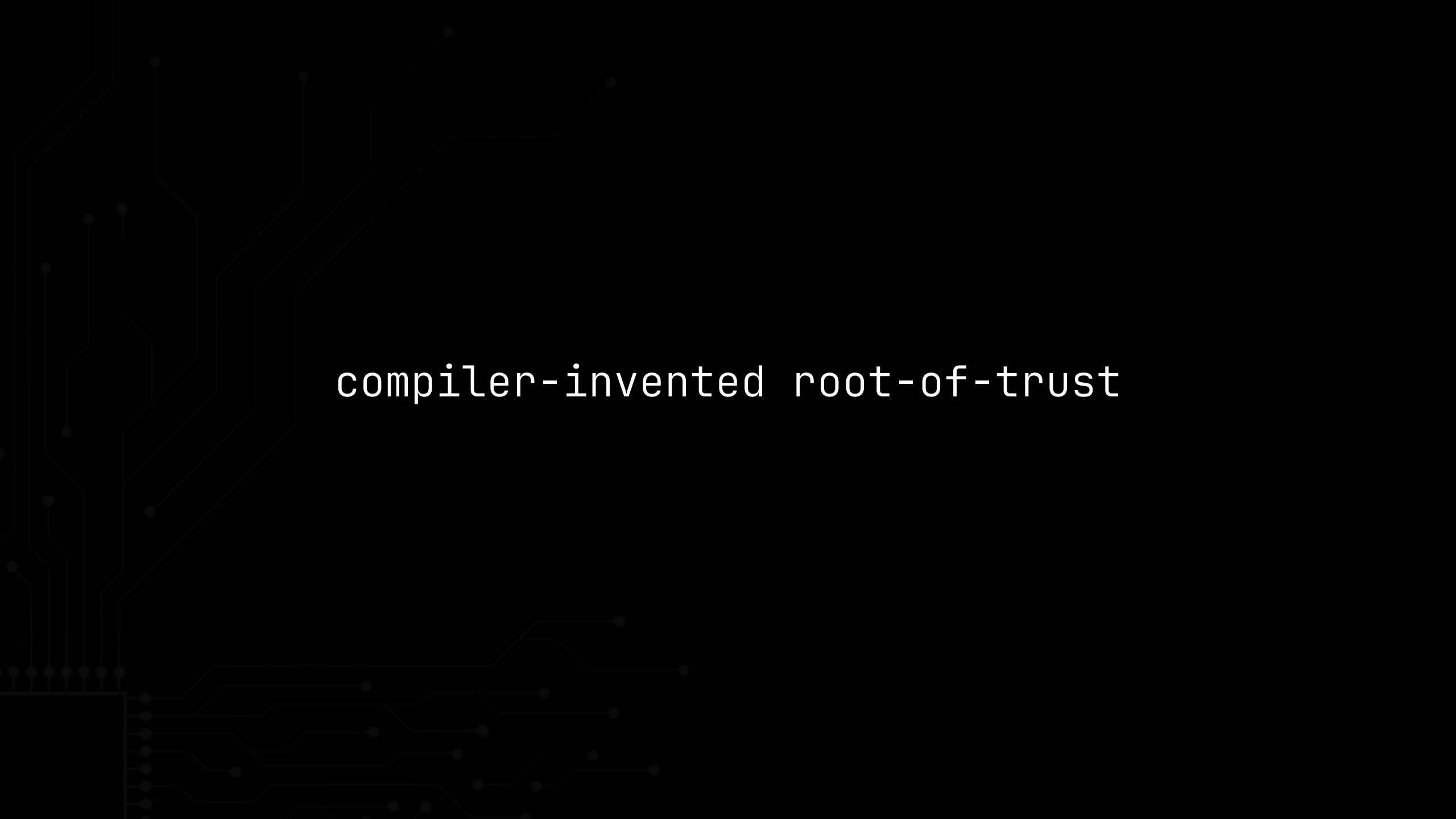

compiler-invented platform persistence

// smm

```
CopyMem (                                          // 199 [SNAPSHOT] copy attacker CommBuffer to stack local
  &TempLockBoxParameterRestore,
  LockBoxParameterRestore,
  sizeof (EFI_SMM_LOCK_BOX_PARAMETER_RESTORE));

if (!SmmIsBufferOutsideSmmValid (                  // 204 [CHECK] .Buffer/.Length must lie outside SMRAM
    (UINTN)TempLockBoxParameterRestore.Buffer,
    (UINTN)TempLockBoxParameterRestore.Length)) {
  DEBUG ((DEBUG_ERROR, "SmmLockBox Restore address in SMRAM or buffer overflow!\n"));
  LockBoxParameterRestore->Header.ReturnStatus = (UINT64)EFI_ACCESS_DENIED;
  return;
}

if ((TempLockBoxParameterRestore.Length == 0) && (TempLockBoxParameterRestore.Buffer == 0)) {
  /* ... */
} else {
  Status = RestoreLockBox (                                  // 220 [USE]
    &TempLockBoxParameterRestore.Guid,
    (VOID *)(UINTN)TempLockBoxParameterRestore.Buffer,       // 222 [USE] dest buffer
    (UINTN *)&TempLockBoxParameterRestore.Length             // 223 [USE] in/out length, writes through
    );
```


compiler-invented root-of-trust

// tpm

```
UINT16          cipherSize = 0;  // size of ciphertext
/* ... */
// Retrieve encrypted data size.
if(UINT16_Unmarshal(                            // 949  [SNAPSHOT] one read of live request
    &cipherSize,
    &buffer,
    &bufferSize) != TPM_RC_SUCCESS)
{
    return TPM_RC_INSUFFICIENT;
}

if(cipherSize > bufferSize)                     // 954  [CHECK] validate against remaining buffer length
{
    return TPM_RC_SIZE;
}
/* ... */
if(session->symmetric.algorithm == TPM_ALG_XOR)
    CryptXORObfuscation(session->authHashAlg, &key.b, nonceCaller,
                        &(session->nonceTPM.b),
                        (UINT32)cipherSize,     // 971  [USE] in-place decrypt LENGTH over live buffer
                        buffer);
```

compiler-invented enclave breaches

// sgx

```
ms_foo_t* ms = SGX_CAST(ms_foo_t*, pms);   // ms -> UNTRUSTED, host-writable       CodeGen.ml:1671
ms_foo_t  __in_ms;
if (memcpy_s(&__in_ms, sizeof(ms_foo_t), ms, sizeof(ms_foo_t)))
    return SGX_ERROR_UNEXPECTED;           // §4.3 stack copy, not a barrier

void*  _tmp_buf = __in_ms.ms_buf;          // [SNAPSHOT] snapshot-once into local  CodeGen.ml:1607
size_t _len_buf = __in_ms.ms_len;          // [SNAPSHOT] host-controlled length    CodeGen.ml:1607/1611

CHECK_UNIQUE_POINTER(_tmp_buf, _len_buf);  // [CHECK] -> if(_tmp_buf &&            CodeGen.ml:2334-2337
                                           //   !sgx_is_outside_enclave(_tmp_buf,_len_buf)) return ...

sgx_lfence();                              // §4.7 CPU-only LFENCE — pins NOTHING  CodeGen.ml:1039

if (_tmp_buf != NULL && _len_buf != 0) {
    _in_buf = (void*)malloc(_len_buf);                      // [USE] alloc size   CodeGen.ml:1304
    if (_in_buf == NULL) { status = SGX_ERROR_OUT_OF_MEMORY; goto err; }
    if (memcpy_s(_in_buf, _len_buf, _tmp_buf, _len_buf)) {  // [USE] count + cap  CodeGen.ml:1309
        status = SGX_ERROR_UNEXPECTED; goto err; }          //  compiler may re-derive _len_buf
}
foo((void*)_in_buf, __in_ms.ms_len);
```


compiler-invented .*

// xen

```
l1gpa = gfn_to_gaddr(guest_l2e_get_gfn(gw->l2e)) +
        guest_l1_table_offset(va) * sizeof(gw->l1e);
if ( !hvmemul_read_cache(v, l1gpa, &gw->l1e, sizeof(gw->l1e)) )
{
    gw->l1e = l1p[guest_l1_table_offset(va)];              // 356 [SNAPSHOT] guest PTE (l1p → guest RAM)
    hvmemul_write_cache(v, l1gpa, &gw->l1e, sizeof(gw->l1e));
}

gflags = guest_l1e_get_flags(gw->l1e);                    // 360 [CHECK] present/rights (guest_walk.c)
if ( !(gflags & _PAGE_PRESENT) )
    goto out;

/* Check for reserved bits. */
if ( guest_l1e_rsvd_bits(v, gw->l1e) )                    // 365 [CHECK] reserved bits (guest_walk.c)
{
    gw->pfec |= PFEC_reserved_bit | PFEC_page_present;
    goto out;
}
/* ... */
    guest_walk_to_gfn → guest_l1e_get_gfn(gw->l1e)        // [USE] frame selection (guest_pt.h)
```

// seL4

```
nodeOffset  = getSyscallArg(4, buffer);                    // 62  [SNAPSHOT] crosses trust boundary
nodeWindow  = getSyscallArg(5, buffer);                    // 63  [SNAPSHOT] crosses trust boundary
/* ... */
if (nodeOffset > nodeSize - 1) {                           // 136 [CHECK] offset within node
    /* ... */ return EXCEPTION_SYSCALL_ERROR;
}
if (nodeWindow < 1 || nodeWindow > CONFIG_RETYPE_FAN_OUT_LIMIT) {  // 144 [CHECK] window range
    /* ... */ return EXCEPTION_SYSCALL_ERROR;
}
if (nodeWindow > nodeSize - nodeOffset) {                  // 152 [CHECK] window fits the node
    /* ... */ return EXCEPTION_SYSCALL_ERROR;
}
destCNode = CTE_PTR(cap_cnode_cap_get_capCNodePtr(nodeCap));
for (i = nodeOffset; i < nodeOffset + nodeWindow; i++) {   // 162 [USE] slot-emptiness loop bound
    status = ensureEmptySlot(destCNode + i);               // 163       (opaque call between iterations)
    /* ... */
}
if ((untypedFreeBytes >> objectSize) < nodeWindow) {       // 203 [CHECK] enough memory for window
    /* ... */ return EXCEPTION_SYSCALL_ERROR;
}
return invokeUntyped_Retype(slot, reset, (void *)alignedFreeRef, newType, userObjSize,
                            destCNode, nodeOffset, nodeWindow, deviceMemory);  // 229-231 [USE]
```

// glibc

```
ElfW(Half) ndx = aux->vna_other & 0x7fff;                // 306  [SNAPSHOT] one read of the mapped ELF
/* In trace mode, dependencies may be missing.  */
if (__glibc_likely (ndx < map->l_nversions))             // 308  [CHECK] gate ndx against table size
  {
    map->l_versions[ndx].hash = aux->vna_hash;           // 310  [USE] ndx as write subscript
    map->l_versions[ndx].hidden = aux->vna_other & 0x8000; // 311  [USE] (re-reads vna_other textually)
    map->l_versions[ndx].name = &strtab[aux->vna_name];  // 312  [USE] writes an attacker pointer
    map->l_versions[ndx].filename = &strtab[ent->vn_file]; // 313  [USE] ndx
  }
```

// git

```
chunk_id = get_be32(table_of_contents);
chunk_offset = get_be64(table_of_contents + 4);          // 121 [SNAPSHOT] one read of the mmap
/* ... terminating-id check ... */
if (chunk_offset % expected_alignment != 0) {            // 127 [CHECK] alignment of snapshot
    /* ... error ... */ return 1;
}
table_of_contents += CHUNK_TOC_ENTRY_SIZE;               // 133 advance the live pointer
next_chunk_offset = get_be64(table_of_contents + 4);     // 134 [SNAPSHOT] next offset
if (next_chunk_offset < chunk_offset ||                  // 136 [CHECK] ordering + in-file bound
    next_chunk_offset > mfile_size - the_hash_algo->rawsz) {
    /* ... error ... */ return -1;
}
for (i = 0; i < cf->chunks_nr; i++) {                    // 143 dup-id loop = register pressure
    if (cf->chunks[i].id == chunk_id) { /* ... */ return -1; }
}
cf->chunks[cf->chunks_nr].id = chunk_id;                 // 151
cf->chunks[cf->chunks_nr].start = mfile + chunk_offset;  // 152 [USE] chunk base ptr from offset
cf->chunks[cf->chunks_nr].size = next_chunk_offset - chunk_offset;  // 153 [USE] chunk size
```

*

Everything is vulnerable ...

... and everything is not.

impact

- Schrödinger's TOCTOU is everywhere
- Explored a *sample, not the boundary*
- Exploitability is *not a property of the source* –
  emergent property of compiler × version × arch × flags
- Both safe *and* vulnerable until built –
  then compiler decides
- One optimizer tweak = TOCTOUs
  in thousands of deployed projects overnight

“

I would very much prefer a compiler switch
that instructs the compiler to not do bloody stupid things like this
instead of marking every other load/store in the kernel with volatile.

”

— Peter Zijlstra · lkml · 2015-06-17

solutions

- Who is the culprit?
  - Blame the code: should have marked it volatile
    - Coder: I wrote what I meant - blame the compiler
  - Blame the compiler: ignored the obvious intent
    - Compiler: it is legal and it is fast - blame the spec
  - Blame the spec: too loose to write secure code
    - Spec: we define effects, not methods - blame the code
- A closed loop - all three are right
- Maybe the problem is C itself
- Treat Schrödinger pattern as de-facto vulnerable

solutions

short-term: code changes

- volatile
  - silently dropped at the first non-volatile parameter
- READ_ONCE(), etc.
  - opt-in, per-load - requires already knowing every bug
- asm volatile("" ::: "memory"), barrier(), etc.
  - position-dependent, unverified, decays as code moves
- Opaque out-of-line call, atomic_read, RO buffers, etc.
  - pays real performance, still guarantees nothing

Short term, this is what we have.

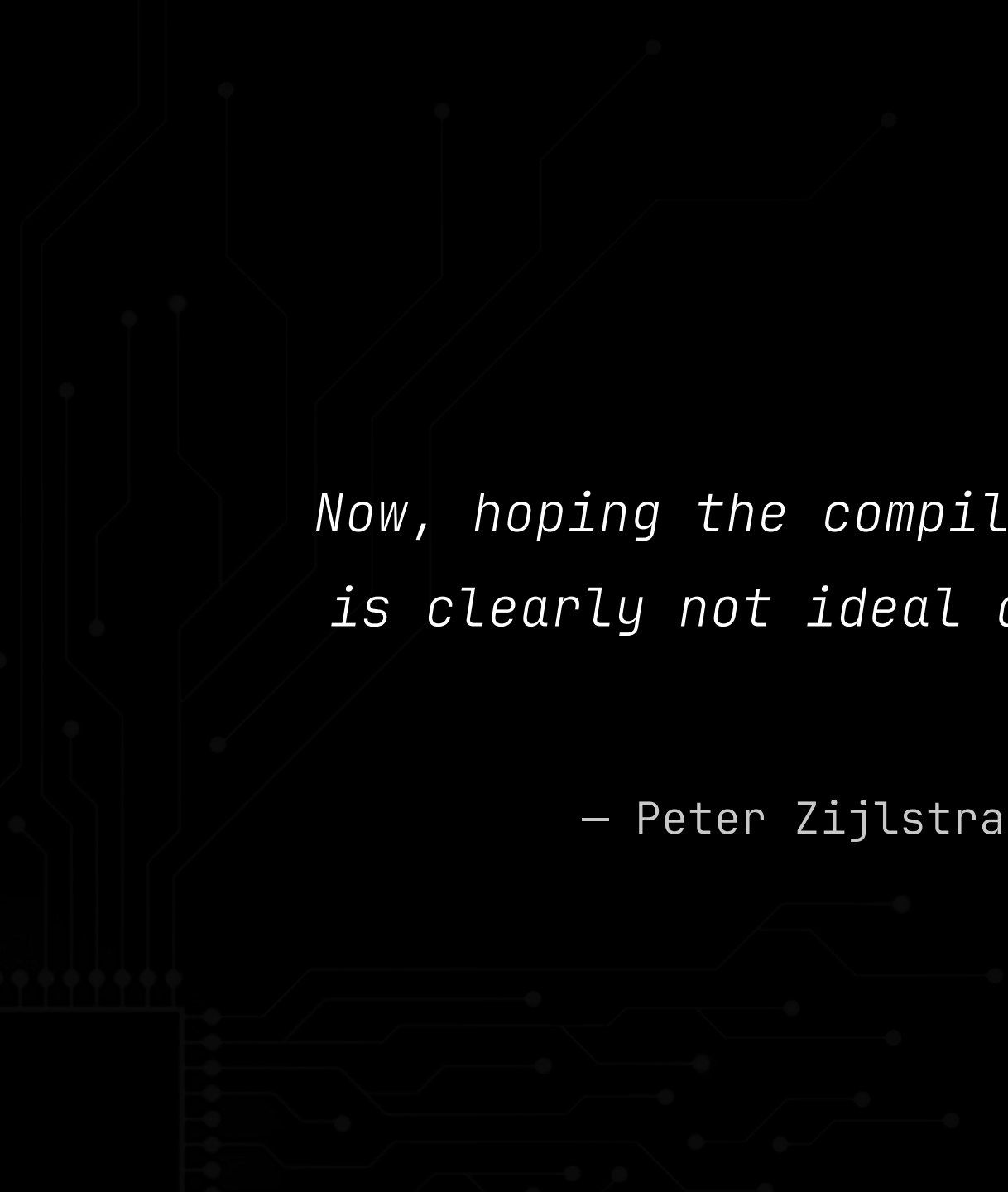
All are manual, unchecked, and fail silently

mid-term: compiler changes

- -fno-invented-loads flag
  - Challenge: pessimizes optimization
- Propagating __untrusted qualifier
  - Challenge: large language and toolchain change

- Memory-model change
- C11 prohibited invented *stores* to shared mem
- Challenge: decade-scale change

long-term: spec changes



> *Now, hoping the compiler generates correct code is clearly not ideal and very dangerous indeed.*

— Peter Zijlstra · lkml · 2020-10-06

implications

```c
typedef struct {
    unsigned int length;
    uint8_t data[];
} packet_t;

uint8_t buffer[MAX_SIZE];

/* kernel handler */
int receive(packet_t* pkt /* userspace data */)
{
    /* copy size locally to prevent TOCTOU */
    unsigned int local_length = pkt->length;
    if (local_length > MAX_SIZE) {
        return -1;
    }
    memcpy(buffer, pkt->data, local_length);
    return 0;
}
```

pkt->length

pkt->length

- Don’t trust the fix.
- Don’t trust the source.
- Don’t trust the build.

implications

"

... the definition of 'sane compiler' grows ever looser.

"

— Paul E. McKenney · lkml · 2024-09-30

try it.

github.com/xoreaxeaxeax/schrodingers-toctou

```c
/* compiler explorer */

unsigned int g(unsigned short *p)
{
    short t = *p;
    return (unsigned short)t - t;
}
```

```asm
; ARM gcc -O2
;   (also: AARCH64/MIPS/MIPS64, -O[gs123])

g:
    ldrh    r2, [r0]   # load *p, once
    ldrsh   r0, [r0]   # load *p, twice
    subs    r0, r2, r0
    bx      lr
```

try it.

C and its Consequences

github.com/xoreaxeaxeax/schrodingers-toctou

Black Hat 2026 · domas · @xoreaxeaxeax